

문제 이해 및 해결 방법

문제 이해

위 코드는 간단한 그래픽 편집기를 구현하는 예제입니다. 사용자는 다양한 형태의 도형(선, 원, 사각형)을 추가하고, 전체 도형을 출력할 수 있는 기능을 제공합니다. 주요 클래스는 Shape라는 추상 클래스를 상속받아 도형을 구현하며, Line, Circle, Rect 클래스가 각각 이를 구체화합니다.

세부 구현 사항

기본 구조 설정: Shape, Line, Circle, Rect 클래스는 각각의 헤더 파일과 소스 파일로 나뉘져 있습니다.

Shape 클래스: 추상 클래스이며, 기본적으로 draw라는 순수 가상 함수를 가지고 있습니다.

Line 클래스: Shape 클래스를 상속받아 draw 함수를 구현합니다.

Circle 클래스: Shape 클래스를 상속받아 draw 함수를 구현합니다.

Rect 클래스: Shape 클래스를 상속받아 draw 함수를 구현합니다.

메인 함수:

`std::vector<Shape*>`를 사용하여 다양한 도형 객체를 저장합니다.

무한 루프를 통해 사용자의 입력을 받아 도형을 추가, 삭제하거나 전체 도형을 출력하는 기능을 제공합니다.

동적으로 할당된 객체는 프로그램 종료 시 메모리 누수를 방지하기 위해 해제합니다.

해결 방법

도형 추가 기능:

사용자 입력에 따라 Line, Circle, Rect 객체를 동적으로 생성하고, `std::vector<Shape*>`에 추가합니다.

각 도형 객체는 Shape 클래스를 상속받아 draw 함수를 구현하므로, 다형성을 활용할 수 있습니다.

도형 출력 기능:

`std::vector<Shape*>`를 순회하며 각 도형 객체의 paint 메서드를 호출합니다.

paint 메서드는 각 도형 클래스에서 draw 함수를 호출하여 해당 도형을 출력합니다.

메모리 관리:

프로그램 종료 시 `std::vector<Shape*>`에 저장된 동적으로 할당된 모든 도형 객체를 해제합니다.

이는 메모리 누수를 방지하고, C++의 RAII(Resource Acquisition Is Initialization) 원칙을 따릅니다